

Sparse Node Perturbation for FPGA-Oriented Online Learning

Progress Report — does sparse (and *adaptive* sparse) really work?

Melike Kara

June 2026

The question we’re answering. *Can adaptive sparse node perturbation reach the same convergence and accuracy as full node perturbation, while cutting the training energy and compute cost in an FPGA-based online-learning system?*

The short answer. At the algorithmic level, sparse perturbation is promising: perturbing a small fraction of neurons preserves most of the accuracy in the tested settings while reducing the active-node perturbation/update cost proxy. However, the *adaptive* part — trying to select the most important neurons — does not outperform simple random or round-robin coverage. The learning signal is distributed fairly evenly across neurons, leaving little for top- k selection to exploit. Rademacher noise also matches Gaussian noise within seed variation in the tested settings.

These results do not yet establish FPGA runtime, resource, power, or energy savings. Vitis HLS implementation and simulation have started, but the hardware baselines are still being validated and are therefore not reported as final results here.

1 How it works, and the two experiment families

(D)ANP in one paragraph. Add a little random noise to the neuron activations, see how much the loss changes, and nudge the weights in the direction that helped:

$$\Delta W_l = N \delta L \frac{\delta a_l}{\|\delta a\|^2} x_{l-1}^\top,$$

where δL is the loss change between the clean and noisy pass and δa_l is how much the activities moved. No backward pass — just two forward passes. DANP adds a decorrelation step that balances the layers; it’s the stronger variant, especially on wide networks and harder data.

Three families of experiments. We have results from two setups; keep them separate when reading the tables:

- **DANP, wide net** [1024, 1024, 1024] (3082 neurons): the main test of sparse-vs-full and of *adaptive* selection (top- k policies), plus the gradient-distribution analysis and the width sweep. MNIST and CIFAR-10.
- **ANP, small net** [64, 64] (138 neurons): the budget sweep down to extreme sparsity, Gaussian-vs-Rademacher noise, and the antithetic test. MNIST and Fashion-MNIST.
- **ANP, SAL validation net** [128, 64]: equivalence testing and analytical delta-propagation MAC accounting for dense linearized vs. sparse-aware linearized ANP, including the new layer-scheduled policy. MNIST and Fashion-MNIST.

Policies. *Random* = a fresh random set of neurons each step. *Scheduled* = round-robin, every neuron visited once per H/k steps (just a counter in hardware). *Adaptive top-k* = pick the k “best” neurons by a score: activity-difference, activity-loss contribution, or gradient-alignment. Optimizer Adam, cross-entropy loss.

2 Does sparse match full?

On the wide DANP network, dropping to 25% active neurons barely moves accuracy — on MNIST it stays within about 0.8% of full, and on CIFAR-10 it’s essentially unchanged (Tables 1, 2). That’s a **4× cut in perturbation/update cost** for a sub-1% accuracy hit.

Table 1: DANP on MNIST, $[1024^3]$, 3 seeds. “Cost” = active/total neurons. **Note the epoch mismatch:** random/scheduled 25% ran 20 epochs, but the adaptive top-k runs ran only 5 — so the fair adaptive-vs-simple comparison is at 50%/5-epochs (all ≈ 0.94), not 25%.

Policy	Fraction	Cost	Final acc	Epochs
Full DANP	—	1.000	0.9642	20
Random	25%	0.250	0.9558	20
Scheduled	25%	0.250	0.9566	20
Random	50%	0.500	0.9407	5
ActivityDiff top-k	50%	0.500	0.9377	5
ActivityLoss top-k	50%	0.500	0.9398	5
ActivityDiff top-k	25%	0.250	0.9313	5
ActivityLoss top-k	25%	0.250	0.9314	5

Table 2: DANP on CIFAR-10, $[1024^3]$, 5 epochs, 3 seeds. All policies ran the same 5 epochs, so this is the clean comparison. All methods converge to about 0.41 under this specific MLP architecture and training setup — and they’re all the same.

Policy	Fraction	Cost	Final acc
Full DANP	—	1.000	0.4094
Random	50%	0.500	0.4080
Random	25%	0.250	0.4070
Scheduled	50%	0.500	0.4122
Scheduled	25%	0.250	0.4102
ActivityLoss top-k	50%	0.500	0.4109
ActivityLoss top-k	25%	0.250	0.4055
GradientAligned top-k	50%	0.500	0.4117
GradientAligned top-k	25%	0.250	0.4076

(Side note: this DANP CIFAR run *converges* at ~ 0.41 . An earlier *plain-ANP* CIFAR pilot on a different, smaller net was unstable and collapsed — so decorrelation is what makes CIFAR behave. See §9.)

3 The effect of “adaptive” smart selection

This is the heart of the research question. The adaptive top-k policies pick neurons by a cleverness score instead of just covering everyone. The result: at matched settings they do **not** beat random or scheduled. On CIFAR (all 5 epochs, fair comparison) every policy lands at ≈ 0.41 — adaptive, random, scheduled, all the same. On MNIST at matched epochs (50%, 5 epochs) it’s the same

story: random 0.9407 vs. activity-diff 0.9377 vs. activity-loss 0.9398 — a wash. The low MNIST 25% top-k numbers (0.931) look worse only because those runs got 5 epochs instead of 20; don’t read them as a real gap.

So the expensive part — ranking/sorting neurons every step — buys nothing.

4 Why adaptive doesn’t help: the signal is diffuse

We measured how concentrated the per-neuron learning signal $g_i \propto \Delta a_i \Delta L / \|\Delta \mathbf{a}\|^2$ is (Table 3). Three numbers say the same thing: the coefficient of variation stays around 0.25–0.28, the Gini coefficient around 0.13–0.16, and the top 10% of neurons carry only ~13–15% of the total signal energy — barely above the 10% you’d get if every neuron were identical.

In plain terms: **no small group of neurons is special**. The useful signal is smeared almost evenly across all of them. If there’s nothing special to find, then “smartly selecting the important neurons” has nothing to exploit — which is exactly why the top-k policies tie with random. The thing that *does* matter is making sure every neuron gets its turn over time, i.e. **coverage**, and a round-robin counter already gives that for free.

Table 3: How concentrated the per-neuron signal is, full DANP [1024³], 5 epochs, mean over 3 seeds. Low CV / low Gini / low top-10% share = diffuse, i.e. no neurons stand out. Stays diffuse throughout training on both datasets.

Epoch	MNIST			CIFAR-10		
	CV	Top10%	Gini	CV	Top10%	Gini
1	0.238	0.135	0.134	0.228	0.132	0.129
2	0.253	0.138	0.142	0.234	0.133	0.132
3	0.263	0.141	0.148	0.238	0.134	0.134
4	0.272	0.143	0.152	0.241	0.134	0.136
5	0.281	0.144	0.156	0.245	0.135	0.138

5 Does it hold across network sizes?

Repeating scheduled-25% across three widths, the accuracy drop stays small everywhere (Table 4). Note the wider net even improves in absolute accuracy — DANP scales up nicely. (Plain ANP, by contrast, got *worse* with width in our earlier runs, which is consistent with an imbalance induced by the global $\|\delta a\|^2$ normaliser; confirming that mechanism would need a dedicated layer-wise analysis. Either way, DANP is the safer base for the big nets.)

Table 4: Width sweep, MNIST, scheduled 25%, 20 epochs, 3 seeds.

Architecture	Cost	Full acc	Sparse acc	Drop
[64, 64, 64]	0.248	0.9391	0.9342	−0.0049
[256, 256, 256]	0.249	0.9530	0.9480	−0.0050
[1024, 1024, 1024]	0.250	0.9642	0.9566	−0.0076

6 Random vs. scheduled, and cheap noise

Random vs. scheduled. On the small net they’re basically tied across fractions (Table 6) — so *how many* neurons matters, *which ones* mostly doesn’t. Scheduled is the more hardware-friendly policy by construction: it can be implemented with a small counter, guarantees coverage,

and avoids per-step ranking or random-mask generation. Its actual FPGA resource and runtime advantage is still to be measured. On the wide $H=1024$ MNIST run the two were also close — their drops relative to full DANP were 0.0084 (random) and 0.0076 (scheduled).

Rademacher = Gaussian. Swapping multi-bit Gaussian noise for $\pm\sigma$ single-bit (Rademacher) noise changes accuracy by less than the seed-to-seed spread (Table 5).

This makes Rademacher noise a promising hardware-oriented replacement for Gaussian noise: it requires only a sign bit rather than a Gaussian sampler, while showing no meaningful accuracy loss in the tested settings. The actual resource and energy difference remains to be measured in hardware.

Table 5: MNIST, ANP [64, 64], scheduled: Gaussian vs. Rademacher (best acc $\pm$ std, $n = 3$). Every difference is within the std.

α	Gaussian	Rademacher	diff
0.0625	0.9257 ± 0.0026	0.9258 ± 0.0023	+0.0001
0.125	0.9266 ± 0.0028	0.9288 ± 0.0005	+0.0022
0.25	0.9262 ± 0.0008	0.9277 ± 0.0028	+0.0015
0.5	0.9299 ± 0.0019	0.9280 ± 0.0013	−0.0019

7 How sparse can we go before it breaks?

MNIST is too easy to show a limit (flat curve, Fig. 1b). Fashion-MNIST shows a real one (Table 6, Fig. 1a): accuracy climbs, then flattens with a clear knee around 12–25%. At 25% we’re at 0.8248, indistinguishable from dense (0.8191); at just 12% we’re within 0.6% of the half-network run — suggesting substantial redundancy in the perturbation budget ($\sim 88\%$ of it can be dropped with little effect here).

Performance only degrades at the most extreme sparsity level. At $\alpha = 0.016$ (approximately three active neurons per step), one seed initially learns normally but collapses late in training (Fig. 2). A plausible explanation is insufficient temporal coverage: with only three active neurons, each neuron is revisited roughly once every 46 steps under the scheduled policy. During these long gaps, the rest of the network continues to change while that neuron’s perturbation-based learning signal is not refreshed. This can make the estimate stale and increase training variance. We therefore interpret the result as evidence of a possible coverage floor: below some minimum active-node budget, neurons are sampled too infrequently for consistently stable learning.

This interpretation is consistent with the broader observation that temporal coverage matters, although the current experiment does not prove that insufficient coverage is the sole cause of the collapse.

Table 6: Fashion-MNIST, ANP [64, 64], Rademacher (best acc $\pm$ std, $n = 3$). Dense baseline 0.8191 ± 0.0036 .

α	Cost	Scheduled	Random
0.016	0.022	0.7863 ± 0.0075	0.7903 ± 0.0045
0.031	0.036	0.8043 ± 0.0017	0.8102 ± 0.0030
0.063	0.065	0.8114 ± 0.0052	0.8140 ± 0.0049
0.125	0.123	0.8147 ± 0.0079	0.8137 ± 0.0040
0.25	0.246	0.8248 ± 0.0035	0.8161 ± 0.0014
0.5	0.500	0.8206 ± 0.0044	0.8228 ± 0.0033

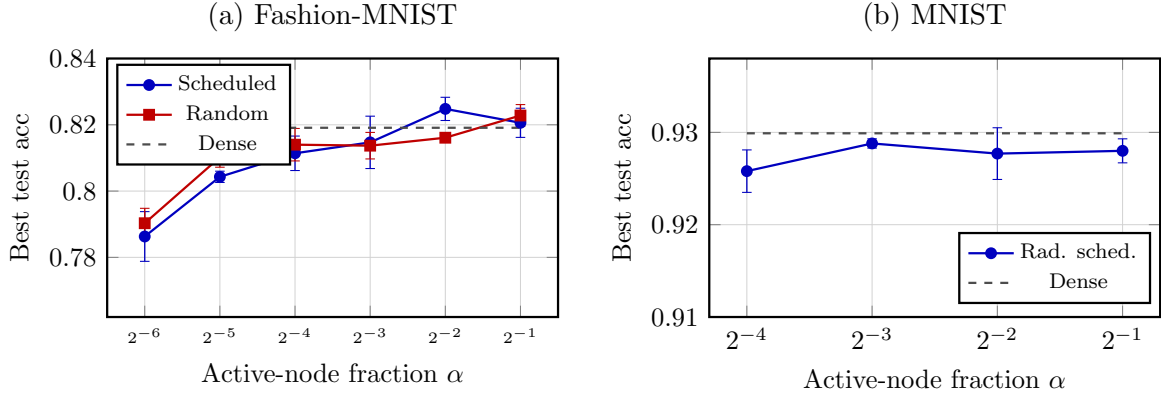


Figure 1: Best test accuracy vs. how many neurons we perturb (± 1 std, $n = 3$), ANP [64, 64]. (a) Fashion-MNIST: a real curve with a knee around 12–25%. (b) MNIST: flat — too easy to stress the method.

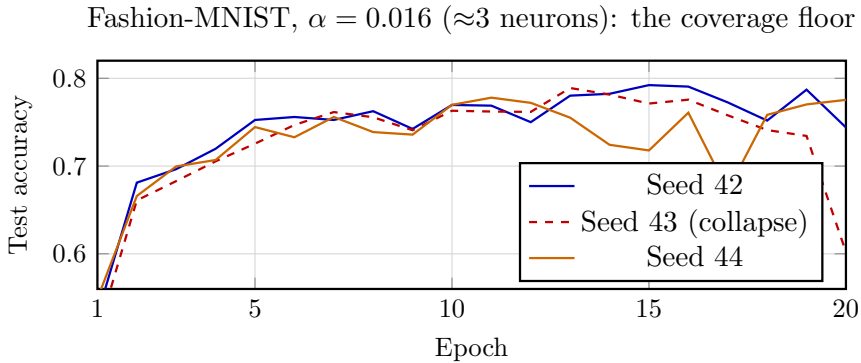


Figure 2: Below the coverage floor: with ≈ 3 active neurons, seed 43 trains fine then collapses at the end. Each neuron is touched only every ~ 46 steps.

8 One trick that didn’t pay off: antithetic sampling

Idea. Use the same noise as $+v$ and $-v$ and average the two estimates; in theory this cancels the curvature error in δL and lowers variance for one extra pass. **Result.** It barely moved (Table 7): antithetic $\approx$ plain single-pass, and the fair 2-pass control (“resample”, two *independent* samples) showed a modest positive trend over it at every tested fraction. **Why.** Our noise is small, so the curvature error antithetic removes is already negligible — it cancels something that barely matters. The real noise at extreme sparsity is first-order sampling variance, which only independent averaging touches. And since both 2-pass modes *double* the perturbation cost — the opposite of our goal — it’s not worth it. We keep single-pass sparse Rademacher.

Table 7: Fashion-MNIST, scheduled Rademacher, extreme regime (best acc $\pm$ std, $n = 3$). Resample beats antithetic everywhere; both cost 2 passes.

α	single (1 pass)	antithetic (2)	resample (2)
0.016	0.7935 ± 0.0038	0.7945 ± 0.0070	0.7987 ± 0.0048
0.031	0.8068 ± 0.0030	0.8046 ± 0.0031	0.8132 ± 0.0032
0.063	0.8076 ± 0.0021	0.8124 ± 0.0037	0.8165 ± 0.0025

9 CIFAR-10: DANP converges, plain ANP didn’t

The DANP CIFAR runs remain stable at approximately 0.41, which is the observed plateau for this MLP architecture and training setup. An earlier plain-ANP pilot on a smaller [256, 256] network

was unstable: the loss diverged and accuracy fell toward chance level. Because the two setups were not fully matched and the plain-ANP run did not converge, this pilot cannot support a direct performance comparison. The contrast nevertheless suggests that decorrelation may contribute to stability on CIFAR-10. Stabilising plain ANP through input standardisation, an ϵ term in the $\|\delta a\|^2$ denominator, and update clipping remains future work rather than an established result.

10 Cutting the algorithm’s own cost (not just the noise)

Everything above shrank the *perturbation* (how many neurons, what noise). The next question is whether the *arithmetic* of an ANP/DANP step has fat to trim — parts of the per-step computation that FPGA hardware pays for but that we can cheapen or skip. Three experiments, each targeting a different piece.

10.1 The normalizer doesn’t need to be exact every step (EMA)

The ANP update divides by a network-wide norm $\|\delta a\|^2$. Computed exactly, that is a global reduction over all active outputs *plus a division*, every single step — and division is expensive in hardware. But that norm changes slowly, so we tried tracking it with a running (EMA) average and refreshing it only every K steps, instead of recomputing it exactly each step:

$$\overline{\|\delta a\|^2} \leftarrow \beta \overline{\|\delta a\|^2} + (1 - \beta) \|\delta a\|_{\text{current}}^2,$$

with $\beta = 0.99$. Each refresh nudges the stored estimate a little toward the current value ($1 - \beta = 1\%$) while keeping most of the old average ($\beta = 99\%$), so it follows the norm smoothly instead of jumping; older values fade out exponentially (hence “exponential”). The update then divides by this stored $\overline{\|\delta a\|^2}$ rather than a freshly computed one. In hardware this also lets you store one reciprocal and *multiply* instead of divide.

This changes nothing about how many neurons are active — it’s orthogonal to sparsity. Only the normalizer computation changes. Table 8 (Fashion-MNIST, ANP [64, 64], Rademacher, scheduled) shows it mostly holds up: at $\alpha = 0.125$ and 0.25 , EMA with $K=100$ matches exact normalization within seed noise — i.e. you can refresh the global norm once every 100 steps and lose nothing. But it’s not bulletproof: at $\alpha = 0.0625$, $K=20$ went unstable (final accuracy collapsed on one seed). So the honest claim is: **exact per-step normalization isn’t always necessary, but EMA isn’t universally safe yet** — it needs more seeds/datasets before we lean on it.

Table 8: EMA vs. exact normalizer, Fashion-MNIST, ANP [64, 64], Rademacher (best acc $\pm$ std, $n = 3$). K = refresh the norm every K steps. Active-node cost is unchanged; only the normalizer differs.

α	mode	K	best acc	min final acc
0.0625	exact	1	0.8161 ± 0.0012	0.7536
0.0625	ema	100	0.8085 ± 0.0050	0.8027
0.0625	ema	20	0.8038 ± 0.0062	0.6108 (unstable)
0.125	exact	1	0.8126 ± 0.0047	0.7991
0.125	ema	100	0.8222 ± 0.0019	0.7968
0.25	exact	1	0.8163 ± 0.0041	0.7980
0.25	ema	100	0.8194 ± 0.0043	0.8155

We did not sweep β : all EMA runs used $\beta = 0.99$. Therefore this is a refresh-frequency ablation over K , not a beta-sensitivity study. Since $\beta = 0.99$ is a slow-moving average, especially when combined with large K , future work should test values such as $\beta \in \{0.9, 0.95, 0.99\}$.

10.2 Linearized ANP preserves learning quality

Standard ANP does a clean forward pass *and* a full nonlinear noisy forward pass, then subtracts. But in the small-noise regime you don’t need the second nonlinear pass at all. The activity difference can be approximated to first order using the derivative you already computed on the clean pass:

$$\delta a_l \approx f'(z_l) \odot (W_l \delta a_{l-1} + \epsilon_l),$$

where $f'(z_l)$ is the clean-pass activation-derivative mask (for Leaky-ReLU, a free 0/1-ish mask) and $\delta a_0 = 0$. So instead of re-running the network through the nonlinearity, you propagate the perturbation forward with cheap linear recursion, reusing the clean masks.

It works. Table 9 (Fashion-MNIST, ANP [64, 64], Rademacher, scheduled) shows linearized ANP matches the full noisy-forward version within seed noise at every budget — differences of +0.0018, −0.0027, −0.0018.

Table 9: Linearized vs. noisy-forward ANP, Fashion-MNIST, [64, 64], Rademacher (best acc $\pm$ std, $n = 3$).

α	noisy forward	linearized	diff
0.0625	0.8094 ± 0.0015	0.8112 ± 0.0045	+0.0018
0.125	0.8174 ± 0.0095	0.8147 ± 0.0042	−0.0027
0.25	0.8210 ± 0.0029	0.8192 ± 0.0013	−0.0018

Compare-mode diagnostic. To directly validate the first-order approximation, we ran `--delta_mode compare` on Fashion-MNIST. Compare mode runs the full nonlinear noisy path and the first-order linearized path on the same batch, same noise, and same sparse mask; training still uses the noisy path, while the linearized path is used only to log approximation metrics. This diagnostic is distinct from the linearized-training experiment in Table 9: there the network is actually *trained* with the linearized path, whereas compare mode keeps the noisy path for the updates and uses the linearized path only to measure the local approximation error.

Table 10: Compare-mode diagnostic for linearized ANP, Fashion-MNIST, ANP [64, 64], Rademacher, scheduled masks, 5 epochs, $n = 3$. Errors are last-epoch means across seeds.

α	rel. δz error	rel. logits error	abs. δL error	rel. δL error	cost
0.0625	0.00161 ± 0.00010	0.00425 ± 0.00045	1.92×10^{-6}	0.0617	0.065
0.125	0.00235 ± 0.00028	0.00661 ± 0.00075	3.41×10^{-6}	0.0958	0.123
0.25	0.00296 ± 0.00028	0.00836 ± 0.00069	4.57×10^{-6}	0.1132	0.246

Honest scope. This does *not* yet remove the matmuls: the delta propagation is still dense in this pilot, so total FLOPs aren’t cut here. What it removes is the nonlinear *activation recomputation* and the need to store a full second set of noisy activations. The “64000 saved” figure we log is just batch $\times$ hidden units = $500 \times (64+64)$ — skipped nonlinear activation evaluations per batch, not FLOPs. The bigger prize (letting sparsity skip delta-matmuls) is exactly what the next subsection does.

This experiment shows that linearized ANP preserves learning quality, but it does not yet prove a substantial hardware speedup. Early HLS results revealed redundant computation in the initial original-ANP baseline, so the hardware comparison is still being refined before final results are reported.

10.3 Sparse-aware linearized ANP (SAL-ANP): turning sparsity into skipped matmuls

Linearized ANP above still runs the delta recursion densely: even when a layer’s δa is exactly zero, the code still computes $W \delta a = W \cdot 0$ and gets zero back. SAL-ANP removes that waste. The key observation is in the recursion itself: δa starts at zero ($\delta a_0 = 0$) and only becomes non-zero once noise is injected. So if perturbation is injected only at one chosen layer s , then every δa before layer s is provably zero, and every matmul that propagates those zeros can be skipped entirely — not computed and discarded, but never computed.

This requires perturbing one layer at a time, so we add a `scheduled_layer` policy: each step selects one eligible layer round-robin, perturbs a fraction α of its nodes, and sets the noise to zero everywhere else. SAL then (i) skips all delta matmuls before the injection layer, (ii) computes δa_s directly from the sparse ϵ_s without the $W \cdot 0$ term, and (iii) uses only the active indices for the first propagation after injection, before δa fills in and the rest of the propagation continues densely.

Question A — is it lossless? (yes) SAL-ANP is a pure computation rewrite: for the same weights, batch, noise, and mask it must produce the *same* δa and the *same* weight update as dense linearized ANP — only the MAC count differs. We verified this with an equivalence test over 4 fractions $\times$ 3 injection layers $\times$ 3 seeds (= 36 cases) on a 784→128→64→10 network. The equivalence itself is analytical: for a fixed model state, batch, perturbation tensor, mask, and selected layer, the skipped prefix terms are exactly zero under the linearized recurrence, so SAL and dense linearized ANP compute the same δa and the same update by construction. The 36 tests are the implementation check on top of that: all passed with **exactly zero** observed difference (`max_abs_delta_diff` = 0, `max_abs_update_diff` = 0), confirming the current code is bit-identical in every tested case. SAL is therefore lossless *under those matched conditions* — it is a computation rewrite, not an approximation.

Table 11: SAL-ANP analytical MAC saving by injection layer, 784→128→64→10, delta-propagation only. Saving is relative to dense linearized delta propagation (282,624 MACs). Later injection layers skip more prefix; the first hidden layer carries the largest matmul, so injecting at or after it already saves most of the cost.

α	injection layer	skipped prefix	MAC saving	sparse MACs
0.25	layer 0 (first hidden)	0	0.696	86,016
0.25	layer 1	1	0.982	5,120
0.25	layer 2 (output)	2	1.000	0
0.50	layer 1	1	0.964	10,240
1.00	layer 0	0	0.000	282,624
1.00	layer 1	1	0.928	20,480

How much is skipped. Table 11 shows where the saving comes from. Almost all of it is skipping the first hidden layer’s matmul: it takes the 784-dim input, so it is by far the largest delta matmul. When injection is at layer 1 or later, that matmul is never run. With round-robin over the three eligible layers, the per-step saving averages to about 0.89 at $\alpha = 0.25$, 0.81 at $\alpha = 0.5$, and 0.64 at $\alpha = 1.0$ (higher α leaves less to skip in the sparse first propagation).

Question B — does layer-scheduling still learn well? This is a separate, empirical question: perturbing one layer per step visits each layer less often, which could hurt via reduced coverage. On MNIST it does not (Table 12): `scheduled_layer` stays in the same ~ 0.91 band as dense-everything, while cutting $\sim 89\%$ of delta-propagation MACs at $\alpha = 0.25$. On Fashion-MNIST it stays in the same accuracy band too (~ 0.79), but with visibly higher seed variance at the

lowest budget — the coverage/stability tradeoff (§4, §7) becoming visible on harder data, not an implementation bug.

Table 12: SAL-ANP learning quality, ANP [128, 64], Gaussian noise, 10 epochs (best acc $\pm$ sample std, $n = 3$). “ δ -MAC saving” is the per-step average delta-propagation MAC reduction from both zero-prefix skipping and sparse first propagation. It is not an end-to-end training-step runtime or energy number.

Dataset	policy	α	best acc	δ -MAC saving
MNIST	scheduled	0.25	0.9123 ± 0.0011	0.696
MNIST	scheduled_layer	0.25	0.9096 ± 0.0054	0.893
MNIST	scheduled_layer	0.50	0.9100 ± 0.0079	0.809
Fashion	scheduled_layer	0.25	0.7869 ± 0.0151	0.893
Fashion	scheduled_layer	0.50	0.7922 ± 0.0098	0.809
Fashion	scheduled_layer	1.00	0.7869 ± 0.0054	0.642

What SAL is and isn’t. Two conclusions should be kept separate. First, SAL-ANP is an exact computational rewrite of dense linearized ANP when the model state, input batch, noise, mask, and injection layer are held fixed. Second, the `scheduled_layer` policy that enables this saving is a separate policy choice.

10.4 Lazy decorrelation for DANP

DANP’s decorrelation has two separate parts: *applying* the matrix R every step (needed for the forward pass) and *updating* R (the expensive $O(D^2)$ – $O(D^3)$ part). Only the update needs to run often — R drifts slowly. So we kept applying R every step but only *updated* it every K steps (`--decor_update_every K`).

Table 13 (CIFAR-10, DANP [256, 256, 256], scheduled 25%, 5 epochs) shows a clean tradeoff: $K=5$ cuts the decorrelation-update frequency to 20% while matching $K=1$ within seed noise. But push it further and it breaks — $K=20$ and $K=100$ clearly degrade. So the lesson is neat and two-sided: **DANP’s decorrelation can be amortized, but not made too infrequent.** (This is DANP-specific; plain ANP has no R to amortize.)

Table 13: Lazy decorrelation, CIFAR-10, DANP [256, 256, 256], scheduled 25%, 5 epochs (best acc $\pm$ std, $n = 3$). K = update R every K steps.

K	update fraction	best acc	vs. $K=1$
1	1.00	0.3902 ± 0.0060	—
5	0.20	0.3868 ± 0.0110	−0.0034 (tie)
20	0.05	0.3609 ± 0.0023	−0.0293
100	0.01	0.3186 ± 0.0086	−0.0716

11 Bottom line — answering the question

- **Sparse matches full.** 25% active neurons keeps accuracy within <1% on MNIST and essentially unchanged on CIFAR, at a $4\times$ cut in the active-node perturbation/update proxy.
- **Adaptive selection does *not* help.** Top-k by any score ties with random/scheduled, because the signal is diffuse (no special neurons). So the answer to “does *adaptive* sparse perturbation help?” is: not on these tasks — and that’s good for FPGA, since it means no ranking/sorting hardware.

- **Coverage is the real lever.** Spreading perturbation over all neurons over time is what matters; a round-robin counter delivers it for free, and the method breaks only below a small coverage floor.
- **Cheaper still:** Rademacher (single-bit) noise matches Gaussian, and the 2-pass variance tricks (antithetic/resample) aren't worth their doubled cost.
- **The algorithm itself has slack too.** Beyond the noise, we cut real per-step arithmetic: the global normalizer can run on an EMA instead of exactly every step (mostly), the second nonlinear noisy pass can be replaced by a first-order linearization with no accuracy loss (strongest result), and DANP's decorrelation update can be amortized ($K=5$ fine, $K \geq 20$ not). All three are still Python-side proxies, not measured FPGA savings.
- **Sparsity finally cuts matmuls.** SAL-ANP turns layer-scheduled sparsity into skipped delta-propagation matmuls, provably lossless vs. dense linearized ANP (36/36 equivalence at exactly zero difference), saving up to $\sim 89\%$ of delta-propagation MACs while holding accuracy in-band on MNIST and Fashion-MNIST.

12 What's next

1. **FPGA cost model on Kria (KR260) — the main next phase.** Turn the Python-side proxies into hardware-relevant numbers: active-node cost, RNG bits (single-bit Rademacher vs. a Gaussian sampler), noisy-pass count, normalizer refresh frequency, DANP R -update frequency, and SAL's skipped-matmul structure $\rightarrow$ estimated LUT/FF/BRAM/DSP, latency, and (if the setup allows) power. This is the unanswered half of the research question — so far everything is an algorithmic proxy. A first concrete step is measuring delta-propagation as a fraction of the whole training step, so SAL's " $\sim 89\%$ " becomes an honest end-to-end number.
2. **Firm up the amortization results:** more seeds/datasets for the EMA normalizer (the K effect is non-monotonic) and a wider K sweep for lazy DANP.
3. **Stabilise plain-ANP CIFAR** (standardise inputs, ϵ in the denominator, clipping) — low priority, since DANP CIFAR already works.